



Git Branching, Merging & Practical Workflow

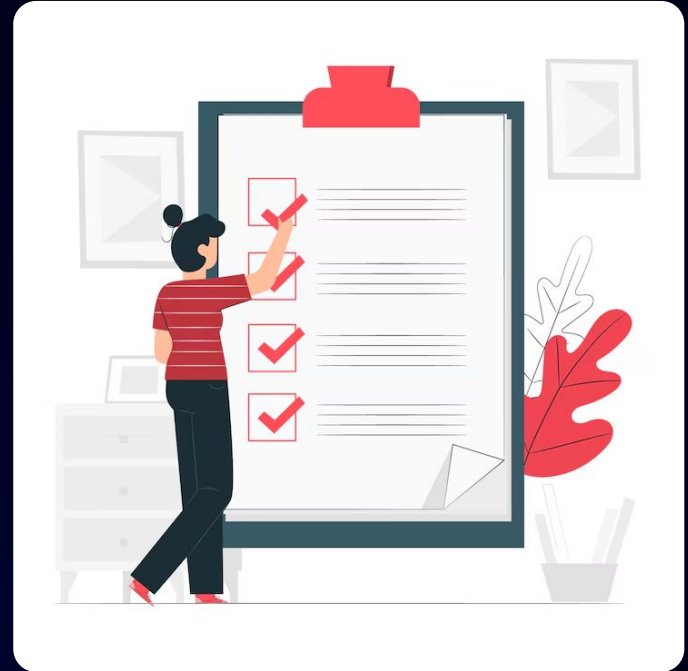
Learning Objectives



- Understand how Git works internally at a deeper level
- Learn and practice branching in Git
- Understand how merging works and how to combine changes
- Use basic recovery commands to fix mistakes
- Apply concepts through a hands-on workflow exercise
- Build confidence in using Git for real-world development

Prerequisites

- Basic understanding of programming concepts
- Familiarity with file and folder structure (directories)
- Basic knowledge of using a computer terminal/command line
- Understanding of how files are created, edited, and saved
- Basic knowledge of Git fundamentals (init, add, commit)
- Awareness of Git architecture (working directory, staging, repository)
- Willingness to learn and experiment with new tools





How Git Works Internally (Simple Mental Model)

How Git Works Internally



Snapshots, Not Differences

Snapshots



- Stores a "full snapshot" of the project for each commit.
- Like taking a picture of your project at each step.
- Helps in tracking the complete state of files over time.

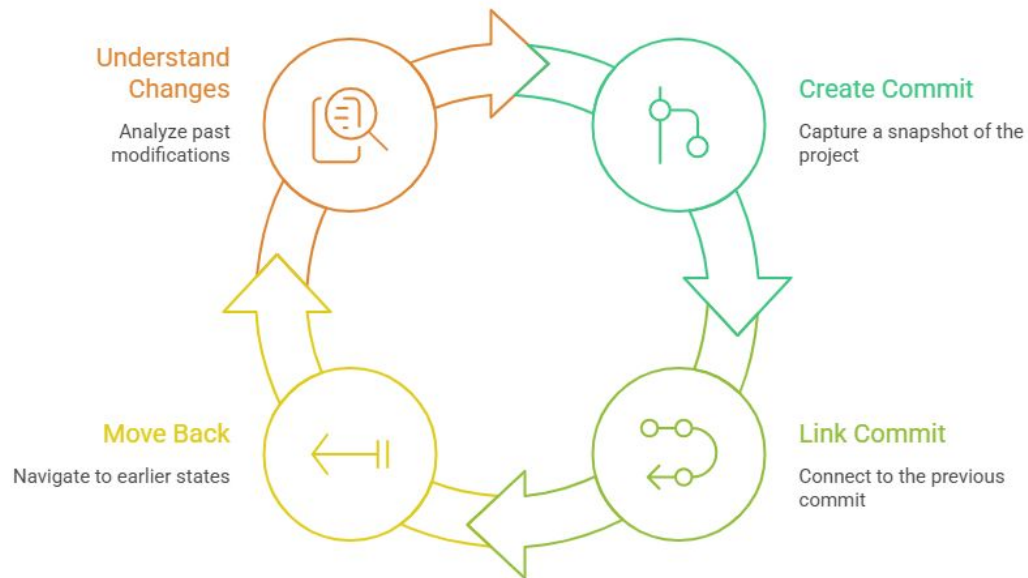
Differences



- Does not store file differences.
- Unlike systems that store file differences.
- (Implied) May require more complex reconstruction of past states.

Commits Form a History Chain

Git Commit History Cycle



Simple Flow

- Workflow:
 - **Staging → Commit → Snapshot**
- Add changes → Commit → Git saves a new snapshot
- No need to worry about internal complexity

Git Workflow Cycle

Stage Changes

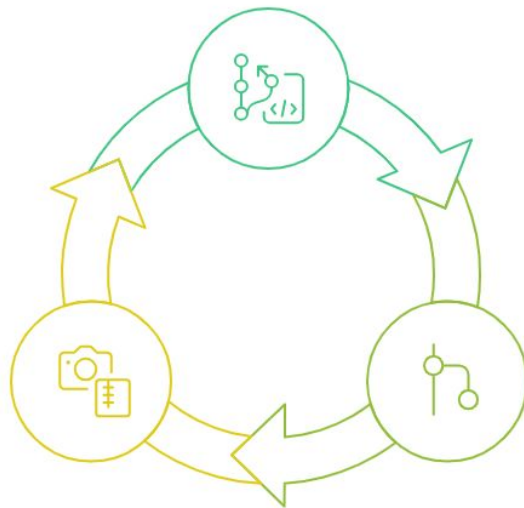
Add modifications to the staging area

Create Snapshot

Git saves a new snapshot of the repository

Commit Changes

Save staged changes as a commit



Pop Quiz

Q. How Git Works Internally

A

As full snapshots of the project

B

As temporary files

Pop Quiz

Q. How Git Works Internally

A

As full snapshots of the project

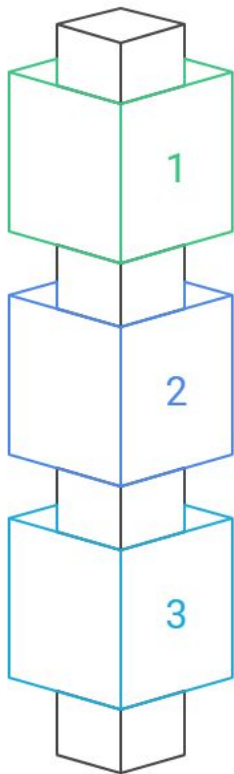
B

As temporary files



Creating Repositories Properly

Creating Repositories



Follow Best Practices

Adhere to established guidelines for Git usage.



Maintain Clean Workflow

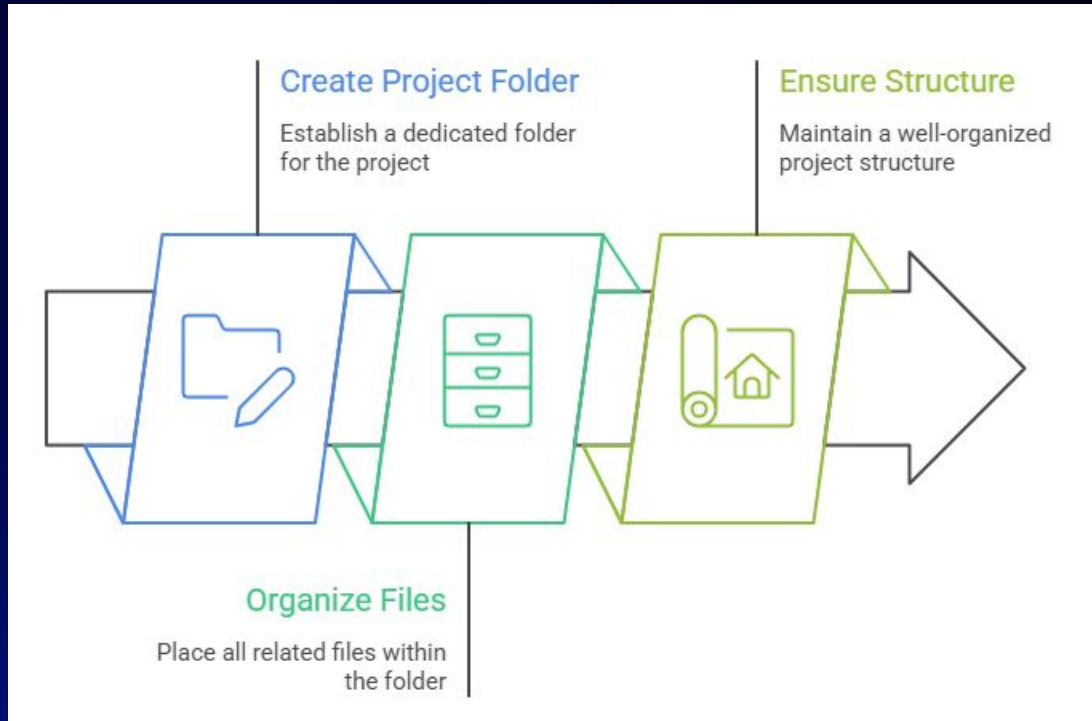
Ensure a streamlined and organized process.



Set Strong Foundation

Create a solid base for effective version control.

STEP 1. Creating a Project Folder

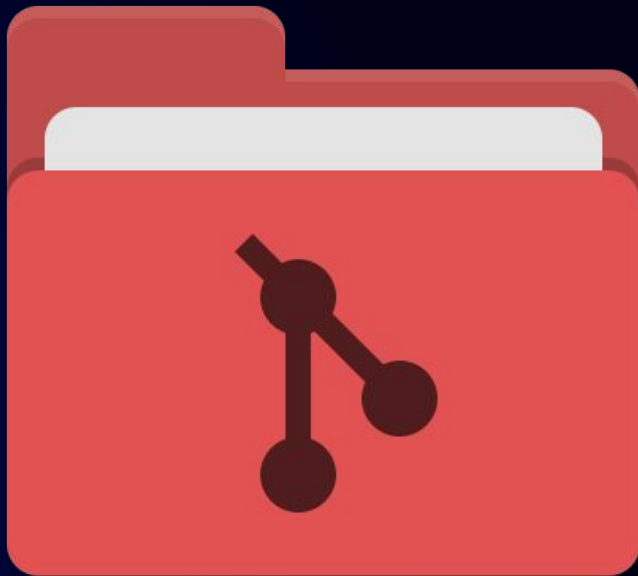


Step 2 – Initialize Git

- Navigate to the project folder
- Run:

`git init`

- This creates a new Git repository

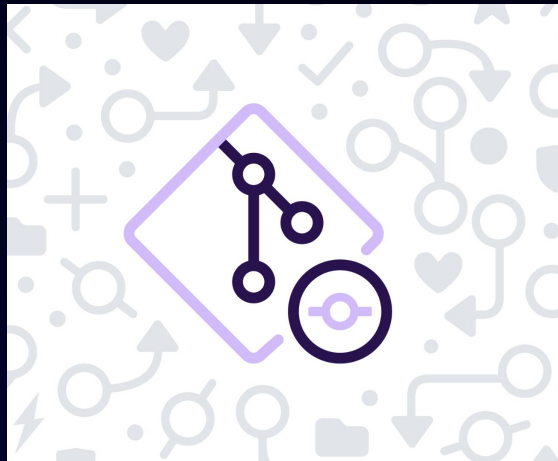


Step 3 – Make First Commit

- Add initial files (e.g., README)
- Stage and commit immediately

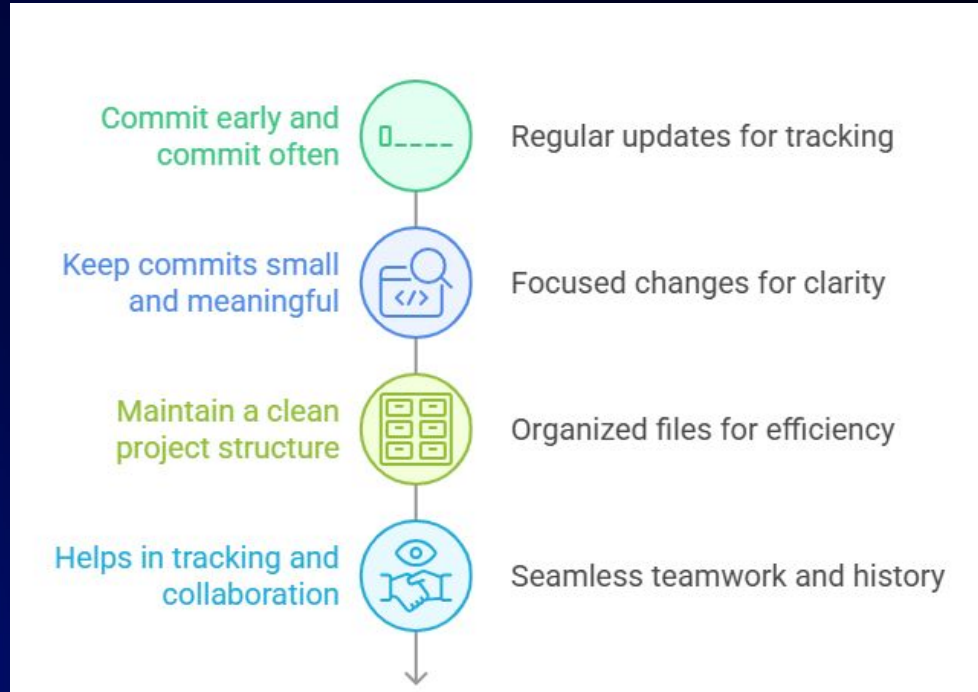
- Example:

```
git add  
git commit -m "Initial  
commit"
```



Best Practices

- Best Practices for Effective Version Control



Pop Quiz

Q. Why should commits be small and meaningful?

A

To avoid collaboration

B

To make history easier to understand

Pop Quiz

Q. Why should commits be small and meaningful?

A

To avoid collaboration

B

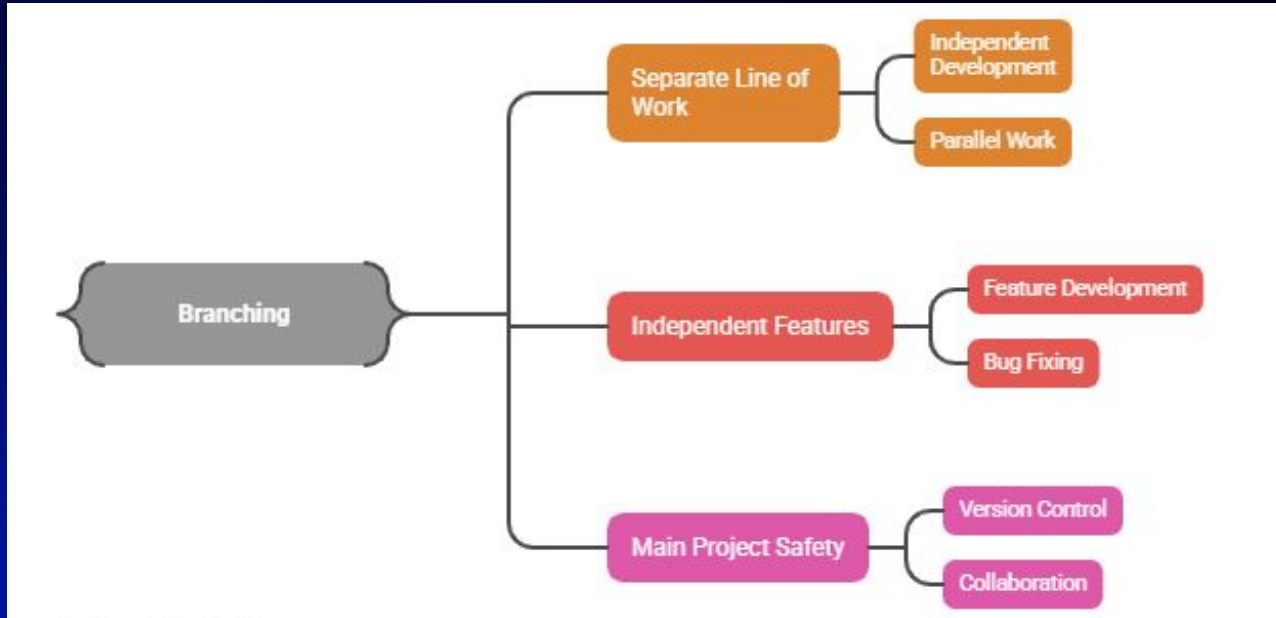
To make history easier to understand



Branching Concept (Core Idea)

What is Branching?

- A branch is a separate line of work
- Allows you to work on features independently
- Keeps your main project safe



Default Branch

- Every repository starts with a default branch
- Usually called main
- Contains the stable version of your project

Default Branch in a Repository



Branching Analogy

Branching Analogy

main

Stable

Production-Ready

branch

Experiment

New Feature

No Effect on main

Why Branching is Important

Enable Safe Changes

Branching allows developers to make changes without risking the main project.

Prevent Breaking the Main Project

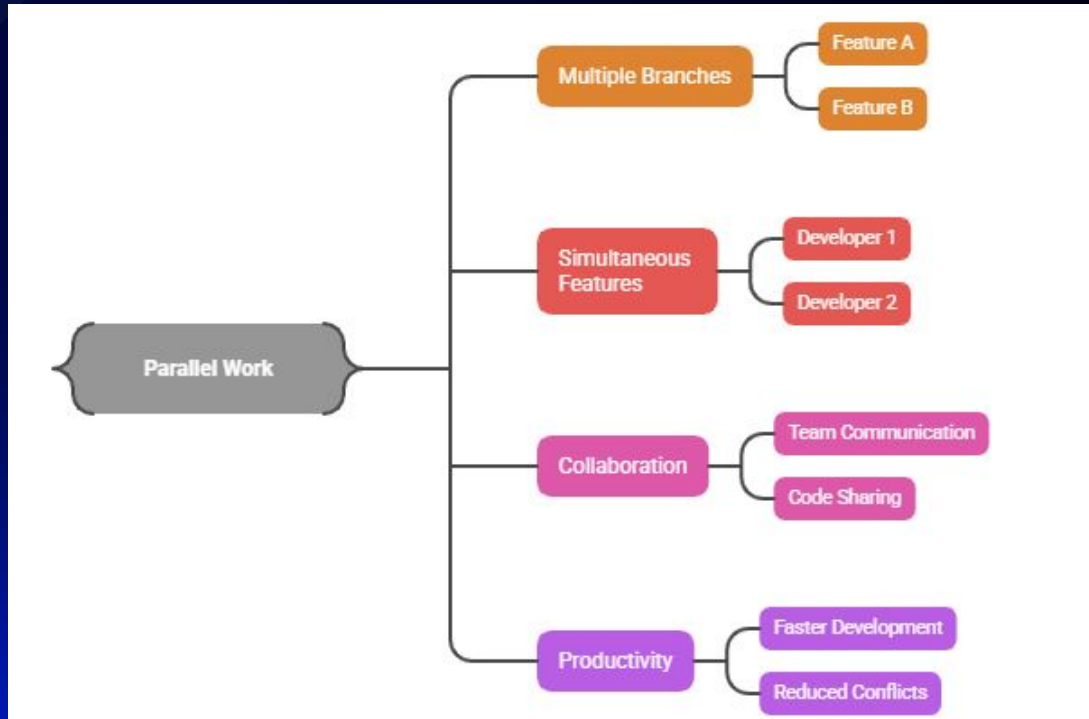
Branches isolate changes, ensuring the main project remains stable.

Help Test New Ideas Easily

Branches provide a sandbox for experimenting with new features.



Parallel Work



Pop Quiz

Q. Can multiple branches exist at the same time?

A

No

B

Yes

Pop Quiz

Q. Can multiple branches exist at the same time?

A

No

B

Yes

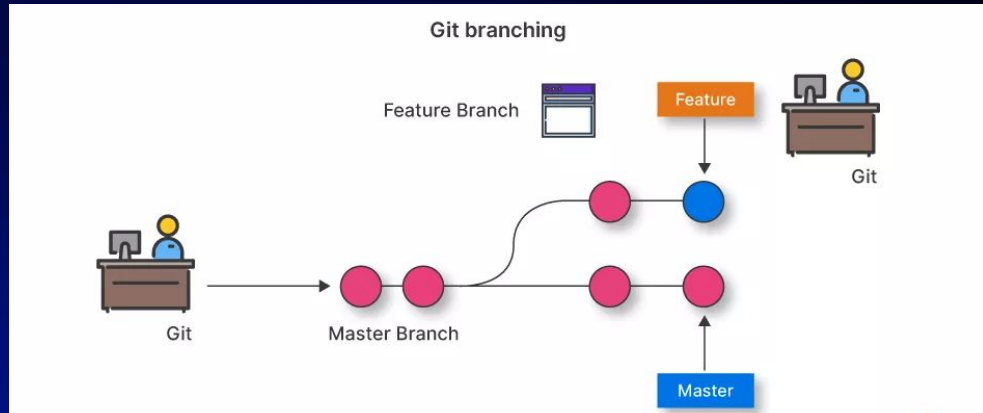


Working with Branches (Hands-On)

Create a Branch

- Create a new branch for your feature
- Command

`git branch feature-1`



Switch to a Branch

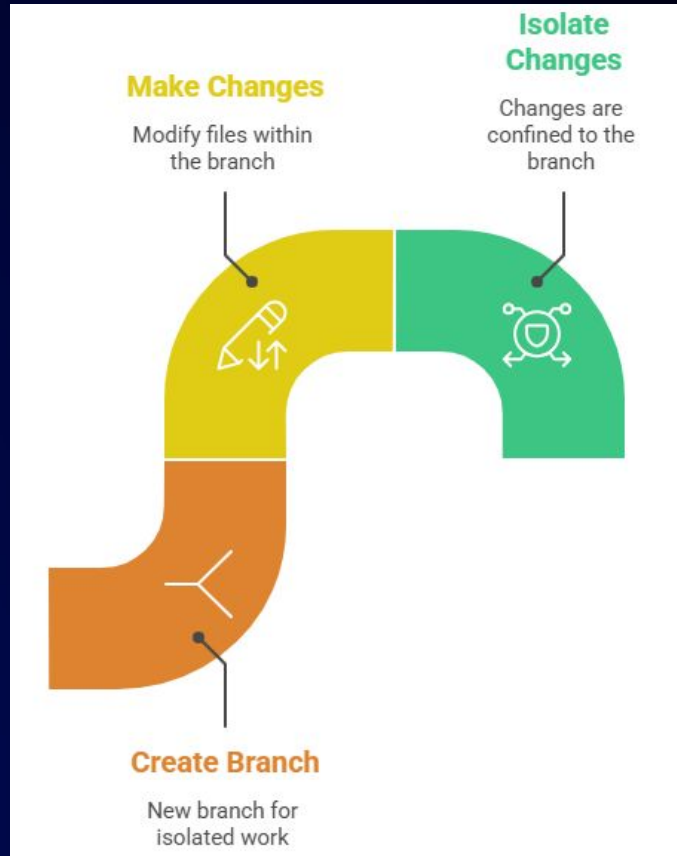
- Move to the created branch
- Commands:

`git checkout feature-1`

Or

`git switch feature-1`

Switch to a Branch



Commit Changes in Branch

- Add and commit your changes:

```
git add .  
git commit -m "Add feature-1 changes"
```

Pop Quiz

Q. After creating a branch using `git branch feature-1`, what happens?

A

The branch is created but you stay on the current branch

B

You automatically switch to the branch

Pop Quiz

Q. After creating a branch using `git branch feature-1`, what happens?

A

The branch is created but you stay on the current branch

B

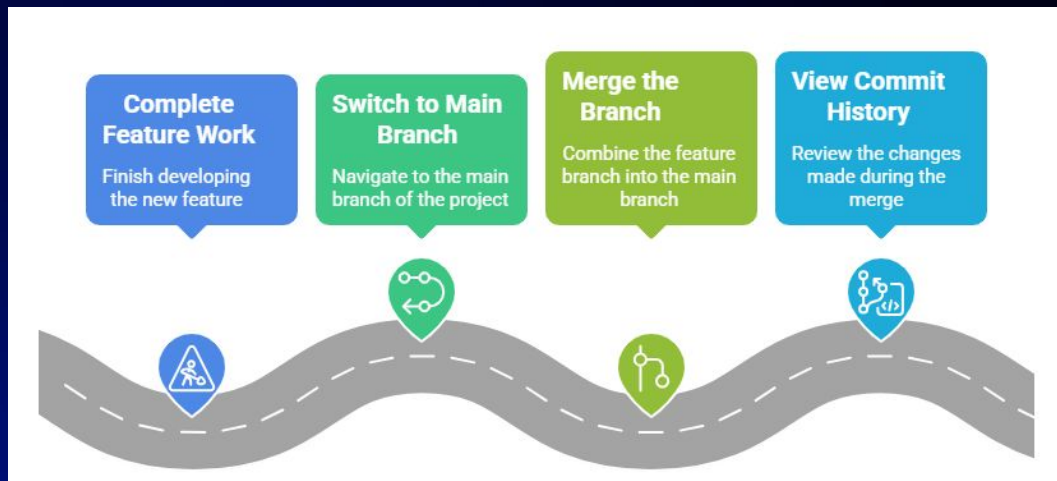
You automatically switch to the branch



Merging Branches

What is Staging?

- Merging combines changes from one branch into another
- Used to bring feature work into the main project
- Common workflow after completing a feature



Switch to Main Branch

- Before merging, go back to the main branch
- Command:

`git switch main`

- Ensures changes are merged into the correct branch

Merge the Branch

- Merge your feature branch into main
- Command:
`git merge feature-1`
- Combines changes from `feature-1` → `main`

What Merge Does

- Integrates changes from another branch
- Keeps history of both branches
- Updates the main branch with new work



Integrates changes



Keeps history



Updates main branch

Pop Quiz

Q. Before merging a branch, what should you do?

A

**Switch to the target branch
(e.g., main)**

B

Commit again

Pop Quiz

Q. Before merging a branch, what should you do?

A

**Switch to the target branch
(e.g., main)**

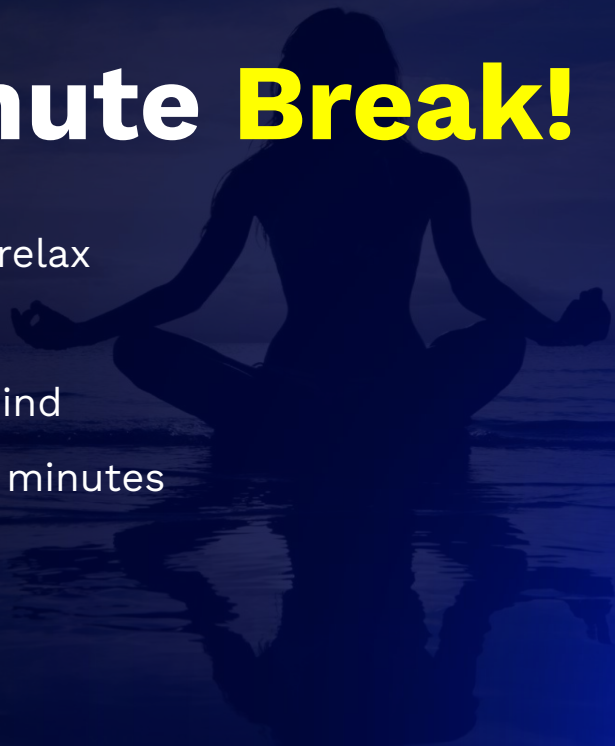
B

Commit again



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes

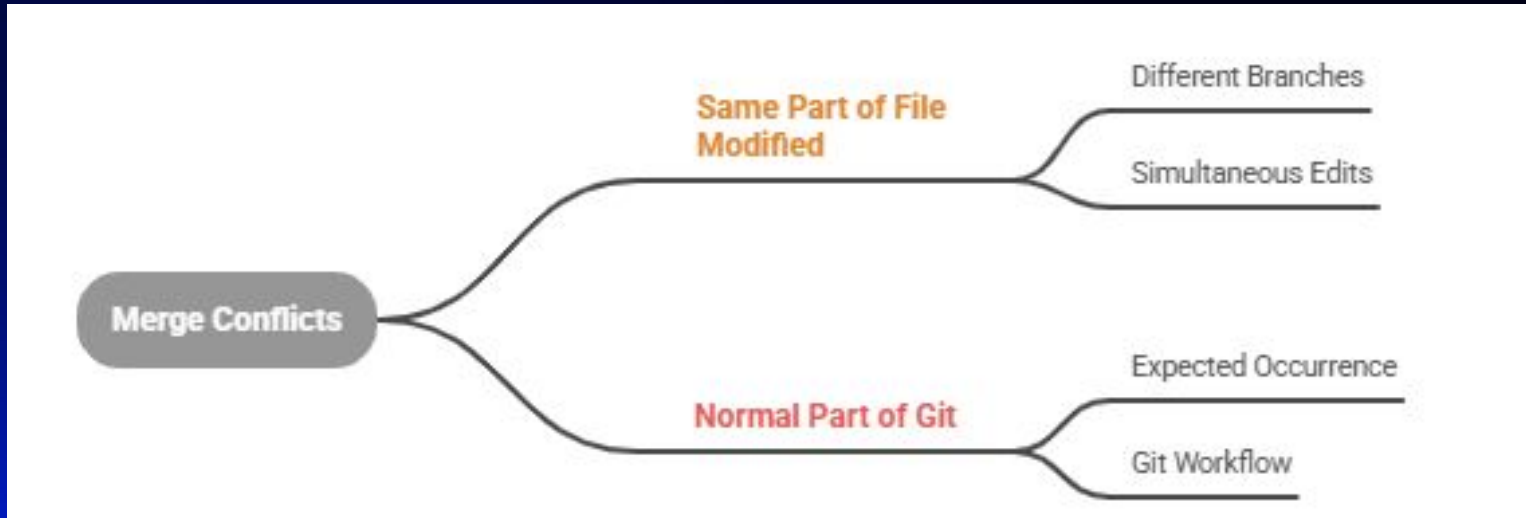




Merge Conflicts (Important)

Merge Conflicts

- Occur when Git cannot automatically merge changes
- Happens when the same part of a file is modified in different branches
- A normal part of working with Git



Conflict Scenario

- Modify the same line in:
 - main branch
 - feature branch
- Try to merge:
`git merge feature-1`
- Git will show a merge conflict

Conflict Markers in Code

- Git highlights conflicts in the file:

```
<<<<<< HEAD  
code from main  
=====  
code from feature-1  
>>>>>> feature-1
```

Shows both versions of the conflicting code

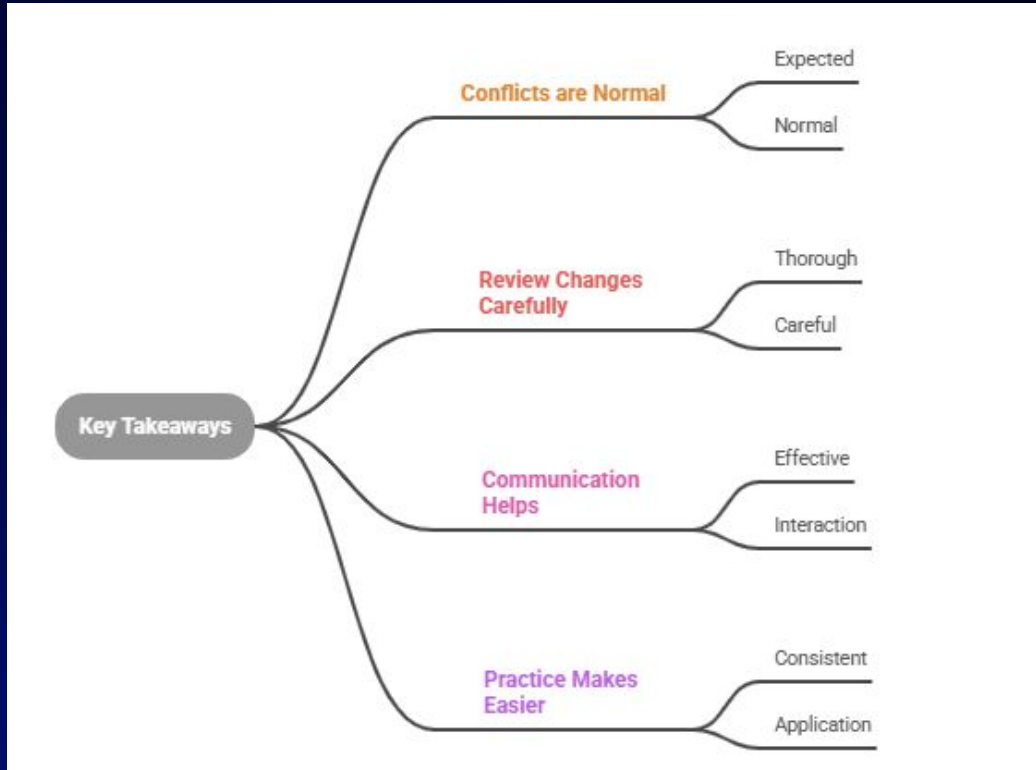
Resolving Conflicts

- Manually edit the file and choose the correct code
- Remove conflict markers
- Then run:

```
git add .  
git commit
```

- This completes the merge

Key Takeaways



Pop Quiz

Q. What does <<<<<< HEAD indicate?

A

Code from another branch

B

Code from the current branch

Pop Quiz

Q. What does <<<<<< HEAD indicate?

A

Code from another branch

B

Code from the current branch

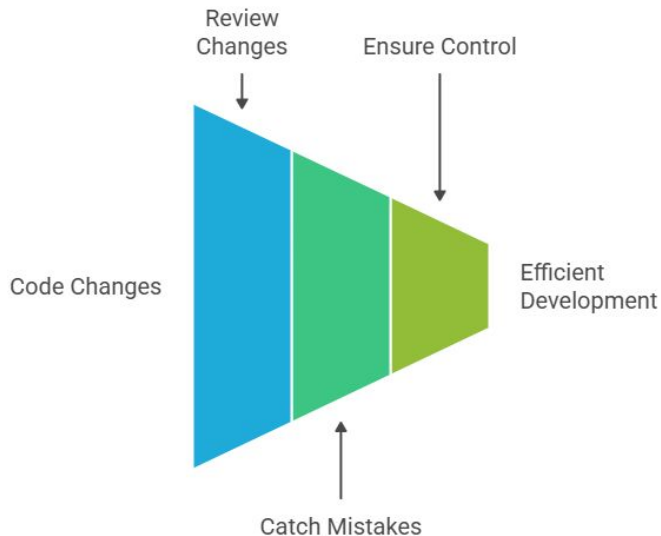


Viewing Changes with git diff

Viewing Changes with git diff

- `git diff` helps you see changes in your files
- Useful to review code before saving (commit)
- Keeps your workflow clean and controlled

Streamlining Code Review with git diff



What is git diff

How to use git diff?



View All Changes

Use ``git diff`` to see all differences between the current working directory and the last commit.



View Specific File Changes

Use ``git diff <filename>`` to see changes in a particular file.



View Changes Between Commits

Use ``git diff <commit1> <commit2>`` to see changes between two commits.

Unstaged Changes

- By default, `git diff` shows unstaged changes
- Changes made but not yet added to staging
- Helps review before using `git add`

Unstaged Changes

- To view staged changes:
`git diff --cached`
- Shows what will be included in the next commit

When to Use git diff



Before Committing

Use `git diff` to review changes before committing to ensure accuracy and avoid mistakes.



Review Work

Use `git diff` to carefully review your work, identifying any errors or omissions.



Avoid Unnecessary Commits

Use `git diff` to prevent committing incomplete or redundant changes.



Debugging

Use `git diff` as a simple but powerful tool for debugging code.

Pop Quiz

Q. Which of the following is true about git diff?

A

It merges branches

B

It only shows differences

Pop Quiz

Q. Which of the following is true about git diff?

A

It merges branches

B

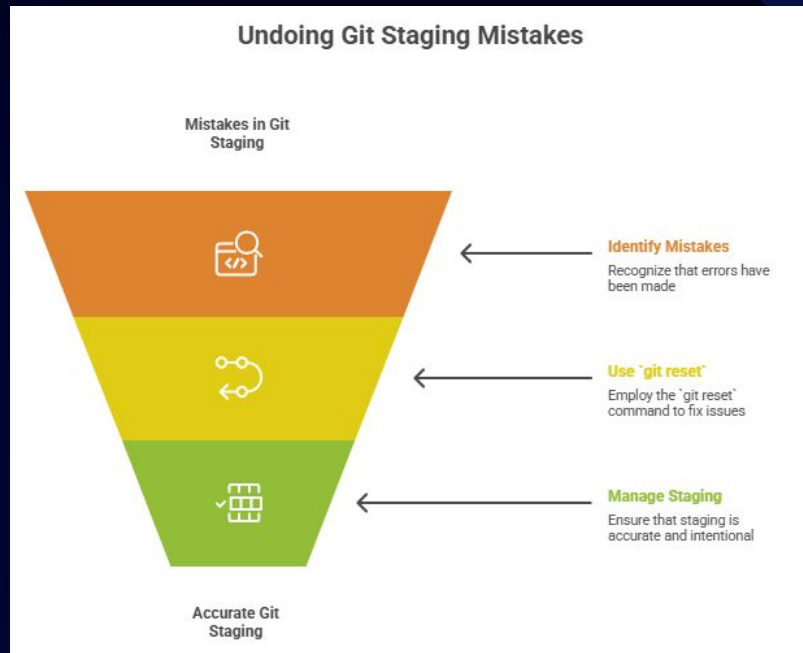
It only shows differences



Undo Basics with git reset

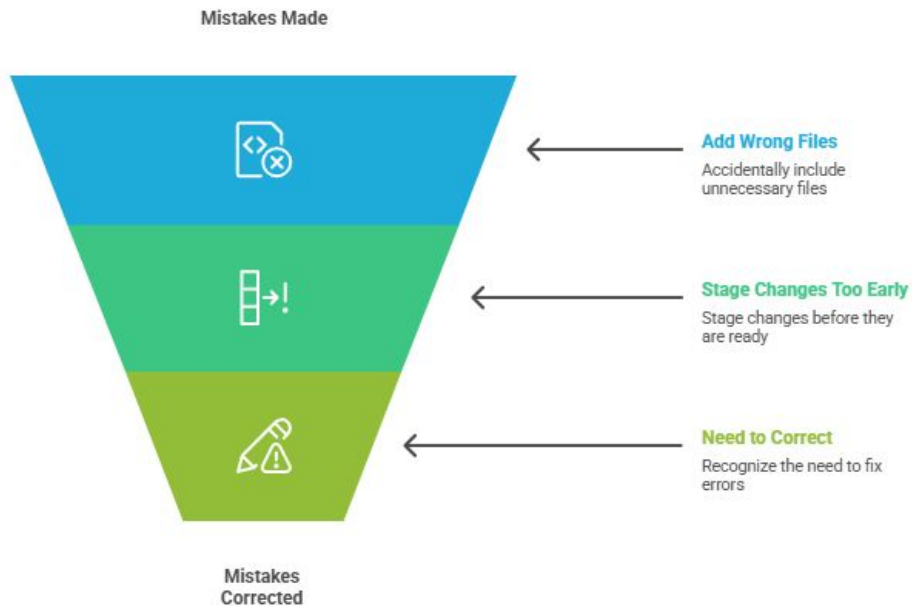
Undo Basics with git reset

- Mistakes happen while working with Git
- Git provides ways to undo changes safely
- **git reset** helps manage and fix staging mistakes



Why Do We Need Undo?

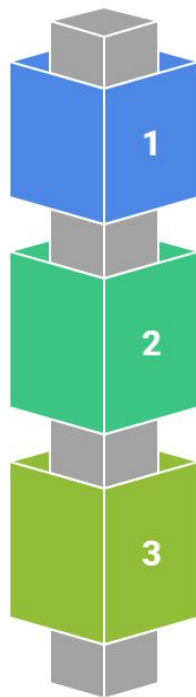
Correcting Mistakes Before Committing



What is git reset?

- Used to undo changes in the staging area
- Moves files from staging → back to working directory
- Basic command:
`git reset`

Undoing Staged Changes



Identify Staged Changes

Recognize the files that have been added to the staging area.



Execute git reset

Run the 'git reset' command to move files back to the working directory.



Files in Working Directory

Confirm that the files are now in the working directory, ready for further modifications.

Unstage Changes

- If you accidentally ran git add
- Use:
`git reset`
- This removes files from staging but keeps your changes

Pop Quiz

Q. What does git reset do in basic usage?

A

**Moves files from staging to
working directory**

B

Pushes changes to remote

Pop Quiz

Q. What does git reset do in basic usage?

A

**Moves files from staging to
working directory**

B

Pushes changes to remote



Temporary Save with git stash

Temporary Save with git stash

Temporarily Save Incomplete Work

Incomplete Work

Need to switch
branches quickly



Use git stash



Complete Work

Work is saved and
restored

Save incomplete work
temporarily

Switch to another
branch quickly

Restore stashed work
when needed

When to Use git stash



Incomplete Feature

Use ``git stash`` to save your current work on a feature without committing it.



Urgent Branch Switch

Use ``git stash`` to quickly switch to another branch without losing your current progress.



Avoid Incomplete Commits

Use ``git stash`` to prevent committing unfinished changes to the main branch.

Save Work Temporarily

- Use:
`git stash`
- Saves your changes and cleans your working directory
- You can now switch branches safely

Restore Stashed Work

- Bring back your saved changes:
`git stash pop`
- Restores changes and removes them from stash

Pop Quiz

Q. Does git stash create a commit?

A

No

B

Yes, Only on main branch

Pop Quiz

Q. Does git stash create a commit?

A

No

B

Yes, Only on main branch

Class Project

Practical Branching Workflow Exercise

Summary

In this lecture, we have covered:

- Understood how Git works internally (snapshots & history)
- Learned proper repository creation practices
- Understood and practiced branching concepts
- Learned merging and handling merge conflicts
- Viewed changes using git diff
- Used git reset for basic undo
- Used git stash for temporary work saving
- Applied concepts through a practical workflow exercise

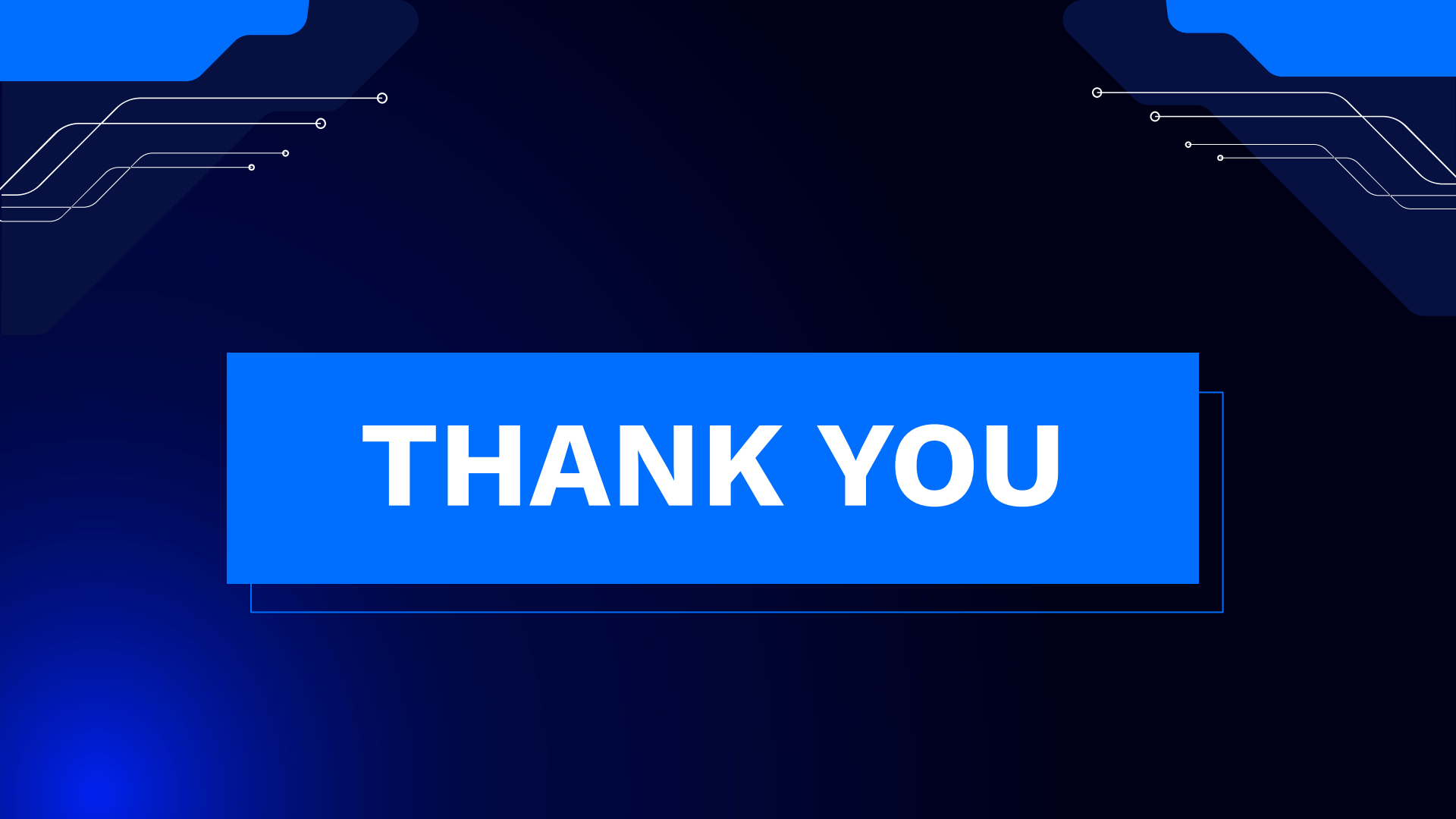




DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU